

Full Project Journal

June 23, 2026

This document is a complete technical journal for the yt-agent2 project: a Streamlit web application that converts any YouTube video into a publish-ready blog post using Groq's Whisper API for audio transcription and Llama 3.3 70B for AI-powered research and writing. It covers system architecture, the full build journey, deployment configuration, known issues, and next steps.

SECTION 1

Project Overview

yt-agent2 is a Python web application that automates the conversion of YouTube videos into publish-ready blog posts. A user pastes a YouTube URL into the Streamlit interface, clicks a button, and within about 60-90 seconds receives a fully written, structured blog post in Markdown — ready to copy, edit, or download.

What the System Does

The pipeline performs three distinct operations automatically: it downloads the audio track from any public YouTube video, transcribes that audio to text using Groq's Whisper model, and then uses an LLM to research the content and write a structured, human-quality blog post from the transcript. No manual effort is required between these steps.

Technology Stack

- **yt-dlp** — downloads YouTube audio streams; supports all public videos
- **ffmpeg** — system-level tool used by yt-dlp to convert audio to MP3 at 64 kbps
- **Groq API (Whisper)** — whisper-large-v3-turbo model; transcribes audio to text at high speed
- **Groq API (LLM)** — llama-3.3-70b-versatile model; powers both the research and writing passes
- **Streamlit** — Python web framework that renders the UI with minimal code
- **python-dotenv** — loads GROQ_API_KEY from a local .env file during development
- **Railway** — cloud deployment platform; hosts the live application
- **Nixpacks** — Railway's build system; installs Python 3.12, ffmpeg, and pip dependencies

Three-Component Architecture

The system is built around three Python classes with clearly separated responsibilities. This separation makes each component independently testable and easy to modify.

Component 1: TranscriptAgent (agents/transcript_agent.py)

Owns the audio download and transcription pipeline. Takes a YouTube URL as input and returns a plain-text transcript string. Internally it: validates the URL, downloads audio via yt-dlp using the 'worstaudio' format selection (to stay within Groq Whisper's 25 MB file limit), post-processes it to MP3 at 64 kbps via ffmpeg, checks the file size, and sends it to Groq's whisper-large-v3-turbo model with `response_format='text'` for a direct string response.

Component 2: BlogPostAgent (agents/blog_post_agent.py)

Owns the content creation pipeline. Takes a plain-text transcript as input and returns a finished blog post as a Markdown string. It makes two sequential LLM calls internally: the first (`_research`) asks Llama 3.3 70B to analyse the transcript and produce structured notes covering core topic, target audience, key insights, hook angle, section structure, human element, and closing direction; the second (`_write`) uses those notes plus the transcript to write a 600-900 word publish-ready post. The research notes are never shown to the user.

Component 3: Orchestrator (*orchestrator.py*)

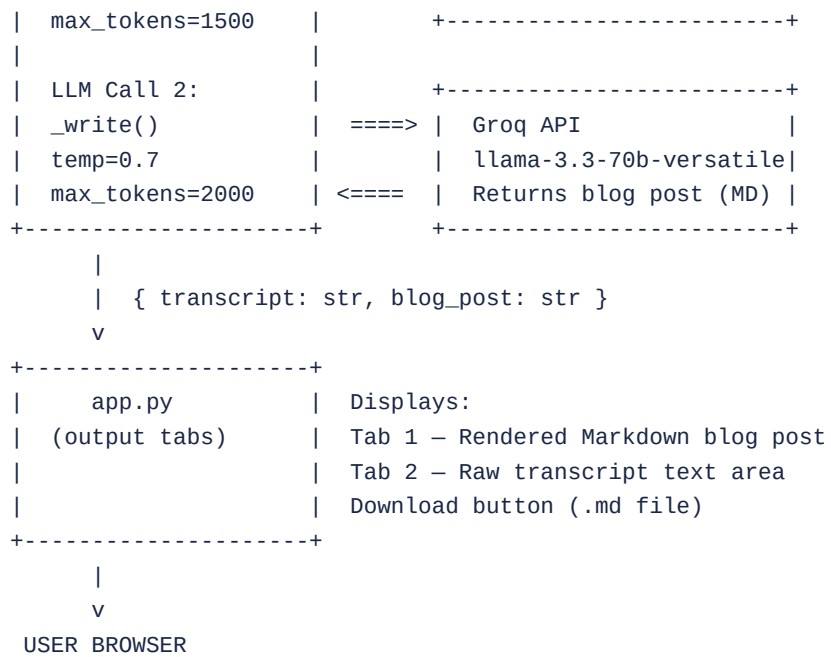
A thin coordinator with no business logic. Instantiates both agents in `__init__`, then in `run()` calls TranscriptAgent first, passes its output to BlogPostAgent, and returns a dict with both 'transcript' and 'blog_post' keys. The Streamlit UI instantiates one Orchestrator per user request.

SECTION 2

Architecture Diagram

The following diagram shows the complete pipeline from user input to final output, including every component, the data that passes between them, and the external APIs involved at each stage.





SECTION 3

Build Journey

This section documents every step taken to build the project, in chronological order. Each step records what was built, what was tested, and what decisions were made along the way.

Step 1

Step 1 — Project Setup

Created the `yt-agent2/` project folder with an `agents/` subdirectory. Created `requirements.txt` listing `yt-dlp`, `groq`, `python-dotenv`, and `streamlit`. Created `.env` and `.env.example` files — the example committed to version control, the real `.env` gitignored. Added `GROQ_API_KEY` to `.env` from the Groq console.

Step 2

Step 2 — TranscriptAgent Built and Tested

Built `agents/transcript_agent.py`. The class takes a YouTube URL, downloads the audio using `yt-dlp` with `'worstaudio/bestaudio'` format and `FFmpegExtractAudio` post-processor (MP3, 64 kbps), checks the file is under 24 MB, then sends it to Groq's `whisper-large-v3-turbo` with `response_format='text'`. Added a `__main__` block for standalone testing. Tested successfully against a 60-second TED-Ed clip (youtube.com/watch?v=arj7oStGLkU).

Step 3

Step 3 — BlogPostAgent Built and Tested

Built `agents/blog_post_agent.py` with two internal LLM calls to `llama-3.3-70b-versatile`. The first (`_research`, `temp=0.4`, `max_tokens=1500`) produces structured notes. The second (`_write`, `temp=0.7`, `max_tokens=2000`) uses those notes plus the transcript to write the blog post. Added a standalone `__main__` test with a hardcoded transcript sample. Tested and verified blog post quality and structure.

Step 4

Step 4 — Orchestrator Built and Tested

Built `orchestrator.py` as a thin coordinator. Instantiates both agents in `__init__`, calls them in sequence in `run()`, and returns a dict with `'transcript'` and `'blog_post'`. Added `__main__` test for the full end-to-end pipeline. Confirmed the full pipeline runs successfully from URL to blog post.

Step 5

Step 5 — Streamlit UI Built

Built `app.py`. UI consists of a URL text input, a primary 'Generate Blog Post' button, and a `st.status()` block that shows live progress messages. Output is displayed in two tabs: 'Blog Post' (rendered Markdown with a .md download button) and 'Raw Transcript' (read-only text area). Used monkey-patching to inject `st.write()` progress feedback into `TranscriptAgent.run()` without modifying the agent.

Step 6

Step 6 — Railway Deployment Files Created

Created three deployment configuration files: Procfile (defines the web process command), railway.toml (sets builder, build command, and start command), nixpacks.toml (installs ffmpeg as a Nix system package during build). All three use \$PORT for the Streamlit server port and 0.0.0.0 for the address.

Step 7

Step 7 — Railway Deployment Attempted, Errors Encountered and Fixed

Pushed to Railway and encountered three deployment errors, each diagnosed and fixed. See Section 6 (Known Issues and Fixes) for the full details of each issue and resolution.

Step 8

Step 8 — Current State: Deployed but Blocked by YouTube Bot Detection

The Railway deployment now builds and starts successfully. The Streamlit UI is accessible from the public URL. However, when a YouTube URL is submitted, yt-dlp receives a 'Sign in to confirm you are not a bot' error from YouTube's servers. This is a known YouTube anti-bot measure that affects server-hosted scrapers. The fix (passing browser cookies) is identified and documented in Section 7.

SECTION 4

Current Status

What is Working

- [OK] Project code is fully written — all three Python components are complete and tested locally
- [OK] Railway deployment pipeline builds successfully (Nixpacks, Python 3.12, ffmpeg, pip install)
- [OK] Streamlit UI starts and is publicly accessible via the Railway-provided URL
- [OK] GROQ_API_KEY is set as a Railway service variable and is correctly read at runtime
- [OK] PYTHONPATH=/app is set in the start command, resolving all module import errors
- [OK] ffmpeg is installed as a Nix package and available in the Railway container
- [OK] The full pipeline works correctly in local development (tested end-to-end)

Current Blocker

[BLOCKED] YouTube bot detection — yt-dlp receives 'Sign in to confirm you are not a bot' when run from Railway's server IP addresses.

YouTube applies bot-detection heuristics based on IP reputation, request headers, and the absence of browser cookies. Cloud server IP ranges (including Railway's) are commonly flagged. yt-dlp can bypass this by using cookies exported from a real browser session — specifically the 'cookies.txt' Netscape-format file that contains the authenticated session cookies YouTube uses to verify a human is browsing.

Next Step

[NEXT] Implement the cookies-based bypass in TranscriptAgent (see Section 7 for full approach)

SECTION 5

File Reference

This section documents every source file in the project. For each file: its location in the folder structure, its purpose in the system, and its complete current contents.

agents/transcript_agent.py

Location

yt-agent2/agents/transcript_agent.py

Purpose

Agent 1. Downloads YouTube audio via yt-dlp (MP3, 64 kbps), checks size against Groq Whisper's 25 MB limit, transcribes via whisper-large-v3-turbo, returns plain-text transcript string.

Full Contents

```
"""
TranscriptAgent
-----
Responsibility: Given a YouTube URL, download the audio and return
a plain-text transcript using Groq's Whisper API.

Dependencies: yt-dlp, groq, python-dotenv
"""

import os
import tempfile
import yt_dlp
from groq import Groq
from dotenv import load_dotenv

load_dotenv()

class TranscriptAgent:
    """
    Agent 1 - Downloads YouTube audio and transcribes it via Groq Whisper.

    Usage:
        agent = TranscriptAgent()
        transcript = agent.run("https://www.youtube.com/watch?v=...")
    """

    # Groq Whisper supports files up to 25 MB.
    MAX_FILE_SIZE_MB = 24

    def __init__(self):
        api_key = os.getenv("GROQ_API_KEY")
        if not api_key:
            raise EnvironmentError(
                "GROQ_API_KEY not found. Add it to your .env file."
            )
        self.client = Groq(api_key=api_key)
        print("[TranscriptAgent] Initialised")

    def run(self, youtube_url: str) -> str:
        self._validate_url(youtube_url)
        print(f"[TranscriptAgent] Processing: {youtube_url}")
        with tempfile.TemporaryDirectory() as tmp_dir:
            audio_path = self._download_audio(youtube_url, tmp_dir)
            transcript = self._transcribe(audio_path)
            print(f"[TranscriptAgent] Done - {len(transcript)} characters transcribed.")
            return transcript

    def _download_audio(self, url: str, output_dir: str) -> str:
        output_template = os.path.join(output_dir, "audio.%(ext)s")
```

```

        ydl_opts = {
            "format": "worstaudio/bestaudio",
            "outtmpl": output_template,
            "postprocessors": [{
                "key": "FFmpegExtractAudio",
                "preferredcodec": "mp3",
                "preferredquality": "64",
            }],
            "quiet": True,
            "no_warnings": True,
        }
        print("[TranscriptAgent] Step 1/2 - Downloading audio ...")
        try:
            with yt_dlp.YoutubeDL(ydl_opts) as ydl:
                ydl.download([url])
        except yt_dlp.utils.DownloadError as e:
            raise RuntimeError(f"Audio download failed: {e}") from e
        audio_path = self._find_audio_file(output_dir)
        size_mb = os.path.getsize(audio_path) / (1024 * 1024)
        print(f"[TranscriptAgent] Audio downloaded - {size_mb:.1f} MB")
        if size_mb > self.MAX_FILE_SIZE_MB:
            raise RuntimeError(
                f"Audio file is {size_mb:.1f} MB, exceeds Groq Whisper's 25 MB limit."
            )
        return audio_path

    def _find_audio_file(self, directory: str) -> str:
        audio_extensions = {".mp3", ".m4a", ".ogg", ".opus", ".wav", ".webm"}
        for filename in os.listdir(directory):
            if os.path.splitext(filename)[1].lower() in audio_extensions:
                return os.path.join(directory, filename)
        raise RuntimeError(
            "No audio file found after download. Check that ffmpeg is installed."
        )

    def _transcribe(self, audio_path: str) -> str:
        print("[TranscriptAgent] Step 2/2 - Transcribing via Groq Whisper ...")
        try:
            with open(audio_path, "rb") as audio_file:
                response = self.client.audio.transcriptions.create(
                    file=audio_file,
                    model="whisper-large-v3-turbo",
                    response_format="text",
                    language="en",
                )
        except Exception as e:
            raise RuntimeError(f"Groq Whisper transcription failed: {e}") from e
        transcript = str(response).strip()
        if not transcript:
            raise RuntimeError("Whisper returned an empty transcript.")
        return transcript

    def _validate_url(self, url: str) -> None:
        if not url or not isinstance(url, str):
            raise ValueError("YouTube URL must be a non-empty string.")
        url = url.strip()
        if "youtube.com" not in url and "youtu.be" not in url:
            raise ValueError(f"'{url}' does not look like a YouTube URL.")

if __name__ == "__main__":
    import sys
    TEST_URL = "https://www.youtube.com/watch?v=arj7oStGLkU"
    print("=" * 60)
    print("TranscriptAgent - standalone test")
    print("=" * 60)
    try:
        agent = TranscriptAgent()
        transcript = agent.run(TEST_URL)
        print("\n--- TRANSCRIPT PREVIEW (first 500 chars) ---")
        print(transcript[:500])
        print(f"\nTotal length: {len(transcript)} characters")
        print("\nTranscriptAgent test PASSED")
        sys.exit(0)
    except Exception as e:
        print(f"\nTest FAILED: {e}")

```

```
sys.exit(1)
```

agents/blog_post_agent.py

Location

yt-agent2/agents/blog_post_agent.py

Purpose

Agent 2. Makes two sequential LLM calls to llama-3.3-70b-versatile: first a structured research pass (temp=0.4), then a writing pass (temp=0.7). Returns a 600-900 word blog post in Markdown.

Full Contents

```
"""
BlogPostAgent
-----

Responsibility: Given a plain-text transcript, run an internal
research pass and then write a polished blog post - both steps
using Groq's Llama 3.3 70B model.

This is ONE agent with TWO internal LLM calls.
There is no separate ResearchAgent.

Dependencies: groq, python-dotenv
"""

import os
from groq import Groq
from dotenv import load_dotenv

load_dotenv()

class BlogPostAgent:
    """
    Agent 2 - Researches a transcript internally, then writes a blog post.

    Internal flow:
    1. _research(transcript) -> research_notes (LLM call 1)
    2. _write(transcript, research_notes) -> blog_post (LLM call 2)
    """

    MODEL = "llama-3.3-70b-versatile"

    def __init__(self):
        api_key = os.getenv("GROQ_API_KEY")
        if not api_key:
            raise EnvironmentError("GROQ_API_KEY not found.")
        self.client = Groq(api_key=api_key)
        print("[BlogPostAgent] Initialised")

    def run(self, transcript: str) -> str:
        self._validate_transcript(transcript)
        print(f"[BlogPostAgent] Received transcript ({len(transcript)} chars)")
        research_notes = self._research(transcript)
        blog_post = self._write(transcript, research_notes)
        print(f"[BlogPostAgent] Done - blog post written ({len(blog_post)} chars)")
        return blog_post

    def _research(self, transcript: str) -> str:
        print("[BlogPostAgent] Step 1/2 - Running internal research ...")
        system_prompt = """You are a senior content researcher and strategist.
Analyse the transcript and produce structured research notes covering:
1. CORE TOPIC, 2. TARGET AUDIENCE, 3. KEY INSIGHTS (4-6),
4. HOOK ANGLE, 5. SECTION STRUCTURE (3-5 headings),
6. HUMAN ELEMENT, 7. CLOSING DIRECTION.
Be specific and concrete."""
        user_prompt = f"Transcript:\n---\n{transcript}\n---\nProduce research notes now."
        try:
            response = self.client.chat.completions.create(
                model=self.MODEL,
                messages=[
                    {"role": "system", "content": system_prompt},
                    {"role": "user", "content": user_prompt},
                ],
            )

```

```

        temperature=0.4,
        max_tokens=1500,
    )
except Exception as e:
    raise RuntimeError(f"Research LLM call failed: {e}") from e
notes = response.choices[0].message.content.strip()
print(f"[BlogPostAgent] Research notes ready ({len(notes)} chars)")
return notes

def _write(self, transcript: str, research_notes: str) -> str:
    print("[BlogPostAgent] Step 2/2 - Writing blog post ...")
    system_prompt = """You are an expert blog writer. Write in clean markdown.
Start with a # Title, use ## for section headings, write in flowing paragraphs.
Target 600-900 words. Never start with a definition. End with a forward-looking
thought. Do not mention YouTube or transcripts."""
    user_prompt = (
        f"Research notes:\n{research_notes}\n\n"
        f"Transcript excerpt:\n{transcript[:3000]}\n\n"
        "Write the complete, publish-ready blog post now."
    )
    try:
        response = self.client.chat.completions.create(
            model=self.MODEL,
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": user_prompt},
            ],
            temperature=0.7,
            max_tokens=2000,
        )
    except Exception as e:
        raise RuntimeError(f"Writing LLM call failed: {e}") from e
    blog_post = response.choices[0].message.content.strip()
    if not blog_post:
        raise RuntimeError("LLM returned an empty blog post.")
    return blog_post

def _validate_transcript(self, transcript: str) -> None:
    if not transcript or not isinstance(transcript, str):
        raise ValueError("Transcript must be a non-empty string.")
    if len(transcript.strip()) < 100:
        raise ValueError("Transcript too short (minimum 100 characters).")

```

orchestrator.py

Location

yt-agent2/orchestrator.py

Purpose

Thin coordinator. Instantiates both agents on startup, calls them in sequence in run(), and returns {transcript, blog_post} to the Streamlit UI. Contains no business logic.

Full Contents

```

"""
Orchestrator
-----
Coordinates TranscriptAgent and BlogPostAgent in sequence.
Receives a YouTube URL, returns a finished blog post.

Flow:
    YouTube URL -> TranscriptAgent -> transcript -> BlogPostAgent -> blog post
"""

from agents.transcript_agent import TranscriptAgent
from agents.blog_post_agent import BlogPostAgent

class Orchestrator:
    """Thin coordinator - holds no business logic of its own."""

    def __init__(self):
        print("[Orchestrator] Initialising agents ...")
        self.transcript_agent = TranscriptAgent()
        self.blog_post_agent = BlogPostAgent()

```

```

        print("[Orchestrator] Ready")

def run(self, youtube_url: str) -> dict:
    """
    Returns:
        dict with keys:
            'transcript' - raw transcript string
            'blog_post' - finished blog post in Markdown
    """
    print(f"\n[Orchestrator] Starting pipeline for: {youtube_url}")
    print("[Orchestrator] Calling TranscriptAgent ...")
    transcript = self.transcript_agent.run(youtube_url)
    print(f"[Orchestrator] Transcript received ({len(transcript)} chars)")
    print("[Orchestrator] Calling BlogPostAgent ...")
    blog_post = self.blog_post_agent.run(transcript)
    print(f"[Orchestrator] Blog post received ({len(blog_post)} chars)")
    print("[Orchestrator] Pipeline complete")
    return {"transcript": transcript, "blog_post": blog_post}

if __name__ == "__main__":
    import sys
    TEST_URL = "https://www.youtube.com/watch?v=arj7oStGLkU"
    print("=" * 60)
    print("Orchestrator - full pipeline test")
    print("=" * 60)
    try:
        orchestrator = Orchestrator()
        result = orchestrator.run(TEST_URL)
        print("\n--- TRANSCRIPT PREVIEW (first 300 chars) ---")
        print(result["transcript"][:300])
        print("\n--- BLOG POST PREVIEW (first 500 chars) ---")
        print(result["blog_post"][:500])
        print("\nOrchestrator test PASSED - full pipeline working")
        sys.exit(0)
    except Exception as e:
        print(f"\nOrchestrator test FAILED: {e}")
        sys.exit(1)

```

app.py

Location

yt-agent2/app.py

Purpose

Streamlit entry point. Renders URL input, primary button, live `st.status()` progress block, and two output tabs (rendered blog post + download button, raw transcript). Monkey-patches `TranscriptAgent.run` to inject progress messages.

Full Contents

```

import streamlit as st
from orchestrator import Orchestrator

st.set_page_config(
    page_title="YouTube to Blog Post",
    page_icon="[YT]",
    layout="centered"
)

st.title("YouTube to Blog Post")
st.caption("Paste a YouTube URL and get a publish-ready blog post in seconds.")

url = st.text_input(
    label="YouTube URL",
    placeholder="https://www.youtube.com/watch?v=...",
    help="Any public YouTube video with speech will work."
)

generate = st.button("Generate Blog Post", type="primary", use_container_width=True)

if generate:
    if not url.strip():
        st.warning("Please paste a YouTube URL first.")
        st.stop()

```

```

try:
    with st.status("Running pipeline...", expanded=True) as status:
        st.write("Initialising agents...")
        orchestrator = Orchestrator()

        st.write("Downloading and transcribing audio...")
        original_run = orchestrator.transcript_agent.run

        def run_with_progress(youtube_url):
            result = original_run(youtube_url)
            st.write(f"Transcript ready - {len(result):,} characters")
            return result

        orchestrator.transcript_agent.run = run_with_progress

        st.write("Researching and writing blog post...")
        result = orchestrator.run(url.strip())

        status.update(label="Done!", state="complete", expanded=False)

except Exception as e:
    st.error(f"Something went wrong: {e}")
    st.stop()

blog_post = result["blog_post"]
transcript = result["transcript"]

st.divider()
tab1, tab2 = st.tabs(["Blog Post", "Raw Transcript"])

with tab1:
    st.markdown(blog_post)
    st.divider()
    st.download_button(
        label="Download as .md",
        data=blog_post,
        file_name="blog_post.md",
        mime="text/markdown",
        use_container_width=True
    )

with tab2:
    st.text_area(
        label="Transcript",
        value=transcript,
        height=400,
        disabled=True
    )

```

requirements.txt

Location

yt-agent2/requirements.txt

Purpose

Python package dependencies with minimum version pins. Used by Railway's build command and local setup. ffmpeg is a system dependency handled by nixpacks.toml.

Full Contents

```

# YouTube to Blog Post - dependencies
# Install: pip install -r requirements.txt

# Core agent dependencies
yt-dlp>=2024.1.0      # YouTube audio download
groq>=0.9.0           # Groq API client (Whisper + Llama)

# Utilities
python-dotenv>=1.0.0   # Load GROQ_API_KEY from .env

# UI
streamlit>=1.35.0      # Web interface

# ffmpeg is also required by yt-dlp for audio conversion.

```

```
# Install separately (not via pip):
#   Ubuntu/Railway: handled by nixpacks.toml (nixPkgs = ["ffmpeg"])
#   Mac:           brew install ffmpeg
#   Windows:       https://ffmpeg.org/download.html
```

Procfile

Location

yt-agent2/Procfile

Purpose

Railway process definition. Binds Streamlit to Railway's injected \$PORT and 0.0.0.0 so the reverse proxy can route external traffic.

Full Contents

```
web: streamlit run app.py --server.port $PORT --server.address 0.0.0.0
```

railway.toml

Location

yt-agent2/railway.toml

Purpose

Railway deployment config. Sets Nixpacks builder, pip install build command, and start command with PYTHONPATH=/app to fix module import resolution.

Full Contents

```
[build]
builder = "nixpacks"
buildCommand = "pip install -r requirements.txt"

[deploy]
startCommand = "PYTHONPATH=/app streamlit run app.py --server.port $PORT --server.address 0.0.0.0"
```

nixpacks.toml

Location

yt-agent2/nixpacks.toml

Purpose

Tells Nixpacks to install ffmpeg (required by yt-dlp for audio conversion) and python312 (pins Python version) as system packages during the build phase.

Full Contents

```
[phases.setup]
nixPkgs = ["ffmpeg", "python312"]
```

.env.example

Location

yt-agent2/.env.example

Purpose

Template showing required environment variables. Committed to version control. The real .env file is gitignored. On Railway, GROQ_API_KEY is set as a service variable in the dashboard.

Full Contents

```
# Copy this file to .env and fill in your key.
# Get your free key at: https://console.groq.com

GROQ_API_KEY=your_groq_api_key_here
```

SECTION 6

Known Issues and Fixes

This section documents each deployment error encountered on Railway, the root cause, and the exact fix applied.

Issue 1 — railway.toml: restartPolicyType rejected by Railway

The initial railway.toml included 'restartPolicyType = "on-failure"' under [deploy]. Railway's current TOML schema does not accept this field and rejected it at deploy time with a configuration parse error.

Fix applied:

Removed the restartPolicyType line entirely. Railway manages restart policy through its dashboard settings, not via the TOML config file.

Issue 2 — ffmpeg not found in Railway container

After the TOML fix, the build succeeded but yt-dlp failed at runtime with 'ffmpeg not found'. The initial nixpacks.toml only listed nixPkgs = ["ffmpeg"] but did not pin the Python version, causing Nixpacks to use a different Python environment that could not find the installed ffmpeg binary.

Fix applied:

Updated nixpacks.toml to nixPkgs = ["ffmpeg", "python312"] to pin Python 3.12 alongside ffmpeg, ensuring both are installed in the same Nix environment.

```
[phases.setup]
nixPkgs = ["ffmpeg", "python312"]
```

Issue 3 — ModuleNotFoundError: No module named 'agents'

The app started but crashed immediately with 'ModuleNotFoundError: No module named agents'. Railway deploys the project to /app inside the container, but Python's module resolution did not include /app in sys.path by default, so the relative import 'from agents.transcript_agent import TranscriptAgent' in orchestrator.py failed.

Fix applied:

Prepended PYTHONPATH=/app to the start command in railway.toml:

```
startCommand = "PYTHONPATH=/app streamlit run app.py --server.port $PORT --server.address 0.0.0.0"
```

Issue 4 (CURRENT) — YouTube bot detection blocks yt-dlp on Railway servers

With all configuration issues resolved and the app running, yt-dlp receives the following error when a YouTube URL is submitted from the deployed app: ERROR: [youtube] ...: Sign in to confirm you're not a bot. This helps protect our community. Root cause: YouTube applies bot detection based on IP address reputation and the absence of authenticated browser session cookies. Cloud provider IP ranges (including Railway's) are heavily flagged. yt-dlp running headlessly with no cookies is indistinguishable from a scraping bot from YouTube's perspective.

Status: Not yet fixed. Fix approach documented in Section 7.

SECTION 7

Next Steps

Fix 1 — YouTube Bot Detection: Cookies Bypass (Immediate Priority)

The standard solution for this class of yt-dlp bot detection is to pass browser cookies from an authenticated YouTube session. yt-dlp accepts cookies in Netscape format via the 'cookiefile' option. The approach:

Step A — Export cookies from a browser

Install the 'Get cookies.txt LOCALLY' Chrome extension (or equivalent for Firefox). Visit youtube.com while signed in to a Google account. Export cookies as cookies.txt in Netscape format.

Step B — Add cookies.txt to the Railway deployment

There are two options: (1) commit cookies.txt to the repo and reference its path in the code — simple but exposes the cookie file; (2) encode cookies.txt as a base64 string, store it as a Railway environment variable (YOUTUBE_COOKIES_B64), and decode it to a temp file at runtime — more secure. Option 2 is recommended.

Step C — Update TranscriptAgent to use the cookies file

Add the cookiefile path to yt-dlp's options dict in `_download_audio`:

```
import base64, tempfile, os

def _get_cookies_path(self) -> str | None:
    """Decode YOUTUBE_COOKIES_B64 env var to a temp file, return its path."""
    b64 = os.getenv("YOUTUBE_COOKIES_B64")
    if not b64:
        return None
    tmp = tempfile.NamedTemporaryFile(
        mode="w", suffix=".txt", delete=False
    )
    tmp.write(base64.b64decode(b64).decode("utf-8"))
    tmp.close()
    return tmp.name

# In _download_audio, add to ydl_opts:
cookies_path = self._get_cookies_path()
if cookies_path:
    ydl_opts["cookiefile"] = cookies_path
```

Fix 2 — Prompt Tuning for Better Blog Post Quality

Once the deployment is unblocked, the next area for improvement is blog post quality. Current areas to tune:

- Increase `max_tokens` in `_write()` from 2000 to 2500 to allow fuller posts without truncation
- Add explicit word-count instruction to the writing system prompt (e.g. 'Write exactly 700-850 words')
- Add a style guide section to the research prompt to preserve voice from the original speaker
- Test with 5-10 different video types (tutorial, interview, talk, explainer) and calibrate per-type
- Consider adding a third LLM call for a brief 'edit pass' to improve opening hook quality

Future Improvements

- **Multi-language support** — remove language='en' from Whisper call and add language detection
- **Progress streaming** — use Streamlit's st.empty() and generator-based LLM streaming to show blog post as it writes
- **Video length guard** — warn users before downloading if video is over 20 minutes
- **Blog post history** — store past generations in session state so users can go back
- **Custom tone selector** — let users choose writing style (professional, casual, technical) before generating
- **SEO metadata** — add a fourth output (meta title, meta description, tags) alongside the blog post
- **Direct publish** — add integrations to post directly to Ghost, Substack, or Medium via their APIs